Court Connect: Technical Document

Oct 15, 2026 · @Yosef

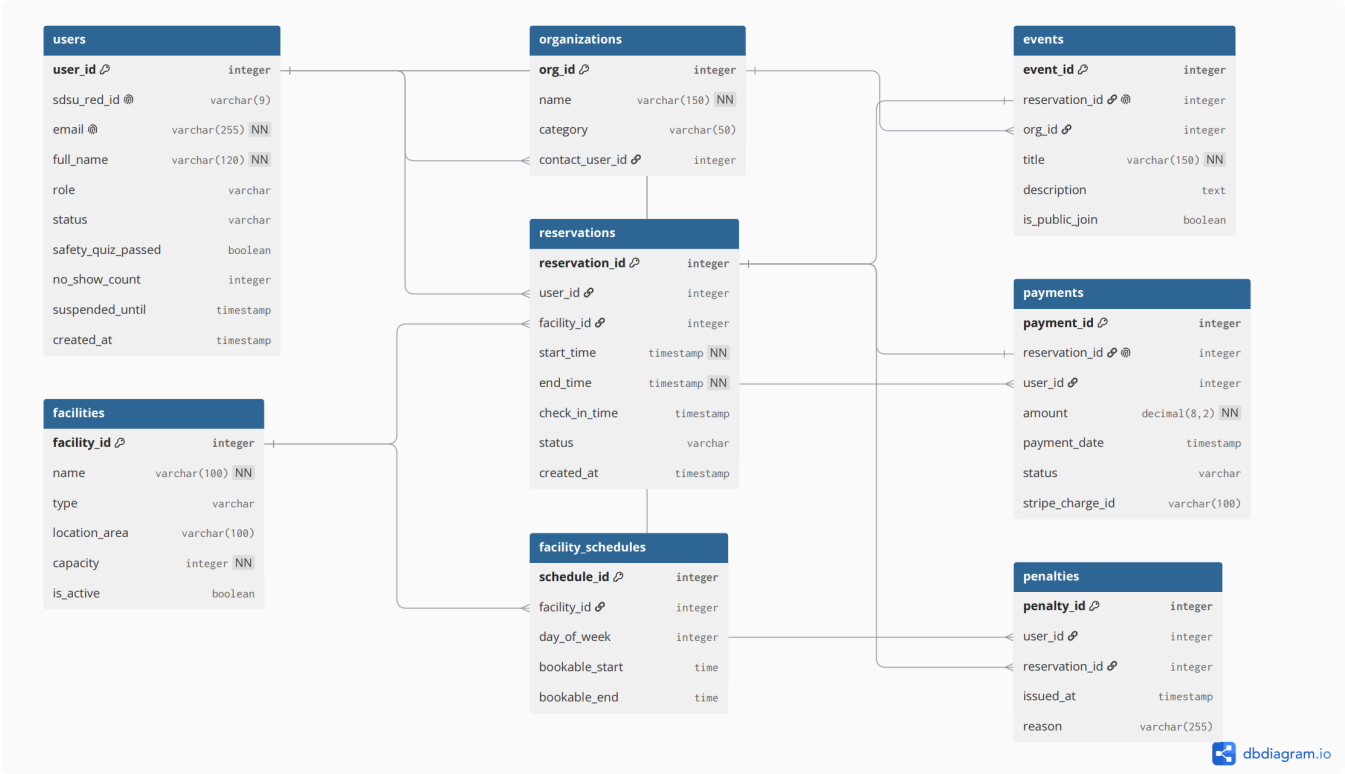
Team	CourtConnect
Members	Yosef Pilip, Riley Kenela, Hudson Mitchell
Course	CS 514, Database Theory and Implementation, San Diego State University
Phase	Draft
Repository	Hmitchell48/CS-514-Project · live site

CourtConnect is one reservation system for every SDSU athletic space: courts, fields, pool lanes, gyms and studios. Students reserve a 2-hour slot and check in on arrival. Clubs book larger spaces and post events. The front-end booking flow is built; the relational database described below is the design it will run on.

1. Design

The database has 10 tables in 3NF. It extends the five-table draft on the project site (Users, Facilities, Reservations, Organizations, Events) with Venues, OperatingHours, OrgMembers, PickupGames and PickupPlayers. Types are written for PostgreSQL.

ER diagram



Entities

- **Users**: students, club contacts, future public users and admins.
- **Venues**: a building or site, such as the Aztec Recreation Center or the Aquaplex.
- **OperatingHours**: a venue's open and close time for each day of the week.
- **Facilities**: one bookable space inside a venue, such as Court 1 or Rec pool lane 2.
- **Reservations**: one user holding one facility for one 2-hour slot.
- **Organizations**: clubs, fraternities and sororities.
- **OrgMembers**: which users belong to which organization.
- **Events**: an organization's booking of a facility, shown on the events feed.
- **PickupGames**: an open game a student posts for others to join.
- **PickupPlayers**: which users have joined which pickup game.

Relationships

Relationship	Cardinality	How it is stored
A venue has many facilities	1 : N	`Facilities.venue_id`

Relationship	Cardinality	How it is stored
A venue has hours for each weekday	1 : N (up to 7)	`OperatingHours.venue_id`
A user makes many reservations	1 : N	`Reservations.user_id`
A facility has many reservations	1 : N	`Reservations.facility_id`
Users belong to many organizations	M : N	junction table `OrgMembers`
An organization has one contact user	N : 1	`Organizations.contact_user_id`
An organization hosts many events	1 : N	`Events.org_id`
A facility hosts many events	1 : N	`Events.facility_id`
An admin approves many events	1 : N	`Events.approved_by`
A user hosts many pickup games	1 : N	`PickupGames.host_user_id`
Users join many pickup games	M : N	junction table `PickupPlayers`

Table schema

PK = primary key, FK = foreign key. Every FK uses `ON DELETE RESTRICT` unless noted, so history is never silently lost.

Users

Column	Type	Key	Constraints
user_id	CHAR(9)	PK	9-digit SDSU RedID
name	VARCHAR(100)		NOT NULL
email	VARCHAR(255)		NOT NULL, UNIQUE, CHECK ends in `@sdsu.edu` for students
password_hash	VARCHAR(255)		NOT NULL
role	VARCHAR(10)		CHECK IN ('student', 'club', 'public', 'admin'), DEFAULT 'student'
status	VARCHAR(10)		CHECK IN ('active', 'warned', 'banned'), DEFAULT 'active'
no_show_count	SMALLINT		NOT NULL, DEFAULT 0, CHECK >= 0

Column	Type	Key	Constraints
locked_until	TIMESTAMP		NULL unless temporarily locked out
quiz_passed_at	TIMESTAMP		NULL until the safety and etiquette quiz is passed

Venues

Column	Type	Key	Constraints
venue_id	VARCHAR(30)	PK	e.g. 'arc', 'aquaplex'
name	VARCHAR(100)		NOT NULL, UNIQUE
location	VARCHAR(150)		NOT NULL

OperatingHours

Column	Type	Key	Constraints
venue_id	VARCHAR(30)	PK, FK → Venues	ON DELETE CASCADE
day_of_week	SMALLINT	PK	CHECK BETWEEN 0 AND 6 (0 = Sunday)
opens_at	TIME		NOT NULL
closes_at	TIME		NOT NULL, CHECK closes_at > opens_at

A day with no row means the venue is closed that day.

Facilities

Column	Type	Key	Constraints
facility_id	VARCHAR(40)	PK	e.g. 'arc-court-1'
venue_id	VARCHAR(30)	FK → Venues	NOT NULL
name	VARCHAR(100)		NOT NULL, UNIQUE (venue_id, name)

Column	Type	Key	Constraints
type	VARCHAR(20)		CHECK IN ('court', 'field', 'pool', 'gym', 'studio', 'other')
detail	VARCHAR(100)		e.g. 'Basketball', 'Lap swim'
capacity	SMALLINT		NOT NULL, DEFAULT 1, CHECK > 0 (students who can hold the same slot)
is_active	BOOLEAN		NOT NULL, DEFAULT TRUE (admin can close a space)

Reservations

Column	Type	Key	Constraints
reservation_id	SERIAL	PK	
user_id	CHAR(9)	FK → Users	NOT NULL
facility_id	VARCHAR(40)	FK → Facilities	NOT NULL
res_date	DATE		NOT NULL
slot_start	SMALLINT		NOT NULL, CHECK IN (7, 9, 15, 17, 19); 11 and 13 are walk-up only
status	VARCHAR(12)		CHECK IN ('booked', 'checked-in', 'no-show', 'cancelled'), DEFAULT 'booked'
created_at	TIMESTAMP		NOT NULL, DEFAULT now()
checked_in_at	TIMESTAMP		NULL until check-in

Partial unique index on (user_id, res_date) WHERE status <> 'cancelled' enforces one reservation per student per day.

Organizations

Column	Type	Key	Constraints
org_id	SERIAL	PK	

Column	Type	Key	Constraints
org_name	VARCHAR(100)		NOT NULL, UNIQUE
type	VARCHAR(12)		CHECK IN ('club', 'fraternity', 'sorority', 'other')
contact_user_id	CHAR(9)	FK → Users	NOT NULL

OrgMembers (junction)

Column	Type	Key	Constraints
org_id	INT	PK, FK → Organizations	ON DELETE CASCADE
user_id	CHAR(9)	PK, FK → Users	ON DELETE CASCADE
member_role	VARCHAR(10)		CHECK IN ('member', 'officer'), DEFAULT 'member'

Events

Column	Type	Key	Constraints
event_id	SERIAL	PK	
org_id	INT	FK → Organizations	NOT NULL
facility_id	VARCHAR(40)	FK → Facilities	NOT NULL
event_date	DATE		NOT NULL
start_slot	SMALLINT		NOT NULL
end_slot	SMALLINT		NOT NULL, CHECK end_slot > start_slot
title	VARCHAR(120)		NOT NULL
description	TEXT		

Column	Type	Key	Constraints
status	VARCHAR(10)		CHECK IN ('pending', 'approved', 'rejected'), DEFAULT 'pending'
approved_by	CHAR(9)	FK → Users	NULL until an admin decides

PickupGames

Column	Type	Key	Constraints
game_id	SERIAL	PK	
host_user_id	CHAR(9)	FK → Users	NOT NULL
facility_id	VARCHAR(40)	FK → Facilities	NOT NULL
game_date	DATE		NOT NULL
slot_start	SMALLINT		NOT NULL
sport	VARCHAR(30)		NOT NULL
max_players	SMALLINT		NOT NULL, CHECK BETWEEN 2 AND 30
notes	VARCHAR(255)		e.g. 'Need 2 more for 5v5'

PickupPlayers (junction)

Column	Type	Key	Constraints
game_id	INT	PK, FK → PickupGames	ON DELETE CASCADE
user_id	CHAR(9)	PK, FK → Users	ON DELETE CASCADE
joined_at	TIMESTAMP		NOT NULL, DEFAULT now()

Normalization

Every table is in third normal form: each non-key column depends on the whole primary key and on nothing else.

- **Venues split out of Facilities.** The front end stores a venue's name, location and hours with every space. In one table, "Aztec Recreation Center, 55th Street" would repeat on all 21 ARC rows, and a change of address would have to touch 21 rows. Since facility_id → venue_id → venue name, that is a transitive dependency. Moving it to Venues (and the hours to OperatingHours) stores each fact once.
- **Members split out of Organizations.** Storing each club's members as a list of RedIDs in one Organizations column breaks 1NF (a multi-valued cell) and can't be joined or constrained. The OrgMembers junction table holds one row per (org_id, user_id), so a student can be in many clubs and each membership is a real foreign key.

2. Implementation

The front end and booking flow are built and live on GitHub Pages; the database and API are the next milestone.

Tech stack

Layer	Choice	Status
Frontend	Plain HTML, CSS and JavaScript, no framework or build step	Built (`index.html` , `js/booking.js` , `css/styles.css`)
Hosting (frontend)	GitHub Pages	Live
Data layer today	Browser `localStorage` plus generated sample data	Built, temporary
Database	PostgreSQL	Planned
Backend	REST API between the page and the database	Planned

Open question: the team has not yet picked the backend framework or database host.

How the pieces connect

The booking page never touches storage directly. Every read and write goes through one `api` object in `js/booking.js` with seven methods: `signUp` , `logIn` , `getDay` , `listReservations` , `reserve` , `cancel` and `checkIn` . Each returns a promise, so

connecting the database means replacing those seven bodies with `fetch()` calls to the backend, which runs the SQL in section 3 against PostgreSQL. Today those methods read and write `localStorage`, and other students' bookings come from a seeded hash so the schedule looks realistic and stays stable between reloads.

Key features built

- **Sign-up wall.** Validates a 9-digit RedID, an `@sdsu.edu` email and an 8+ character password. Two test accounts log in until the Users table exists.
- **Availability view.** Pick one of 8 venues and one of the next 7 days. Each space shows its seven 2-hour slots (7 am to 9 pm) as Open, N spots left, Booked, Full, Walk-up, Event, Closed or Ended.
- **Real facility data.** 74 spaces across 8 venues, with fall 2026 hours from arc.sdsu.edu, including per-space hours for the climbing walls.
- **Reservations.** Confirm dialog, then booking. Enforces one reservation per day, capacity per slot, walk-up windows, venue hours and no booking a slot that has started.
- **Check-in and no-shows.** Check-in opens 15 minutes before a slot and closes at its end. A booking that ends without check-in shows as a no-show.
- **Cancel.** Allowed until the slot starts; the slot frees up for others.
- **Club events.** Sample events (Spikeball, Volleyball, Swim and Soccer clubs) block their slots and appear under the space name.

Not built yet: pickup game posting and alerts, event posting by clubs, and the admin view. Their tables are in the design above.

3. Course Concepts Applied

Each rule the front end checks today in `api.reserve` moves into the database as a key, constraint, trigger or transaction, so no client can bypass it.

Primary and foreign keys

Every table has a primary key: natural keys where one exists (RedID for Users, slugs like 'arc-court-1' for Facilities), SERIAL surrogates elsewhere, and composite keys on the junction tables, e.g. `PRIMARY KEY (game_id, user_id)` on PickupPlayers, which also stops a student joining the same game twice. Foreign keys tie every reservation, event and pickup game to a real user and facility.

```
CREATE TABLE Reservations (
  reservation_id SERIAL PRIMARY KEY,
  user_id        CHAR(9)     NOT NULL REFERENCES Users(user_id),
  facility_id    VARCHAR(40) NOT NULL REFERENCES Facilities(facility_id),
  res_date       DATE        NOT NULL,
  slot_start     SMALLINT    NOT NULL CHECK (slot_start IN (7, 9, 15, 17,
19)),
  status         VARCHAR(12) NOT NULL DEFAULT 'booked'
                 CHECK (status IN ('booked', 'checked-in', 'no-show',
'cancelled')),
  created_at     TIMESTAMP   NOT NULL DEFAULT now(),
  checked_in_at  TIMESTAMP
);
```

SQL queries (SELECT, INSERT, UPDATE, DELETE)

```
-- SELECT: a student's upcoming reservations (backs listReservations)
SELECT r.reservation_id, f.name, v.name AS venue, r.res_date, r.slot_start,
r.status
FROM Reservations r
JOIN Facilities f ON f.facility_id = r.facility_id
JOIN Venues v     ON v.venue_id    = f.venue_id
WHERE r.user_id = '800000001' AND r.res_date >= CURRENT_DATE AND r.status <>
'cancelled'
ORDER BY r.res_date, r.slot_start;

-- INSERT: book a slot (backs reserve)
INSERT INTO Reservations (user_id, facility_id, res_date, slot_start)
VALUES ('800000001', 'arc-court-1', '2026-10-20', 17);

-- UPDATE: check in (backs checkIn)
UPDATE Reservations
SET status = 'checked-in', checked_in_at = now()
WHERE reservation_id = 42 AND user_id = '800000001' AND status = 'booked';

-- DELETE: leave a pickup game
DELETE FROM PickupPlayers WHERE game_id = 7 AND user_id = '800000001';
```

Cancelling is an UPDATE to `status = 'cancelled'`, not a DELETE, so no-show and usage history is kept.

Joins, aggregates, subqueries

```
-- Join + aggregate: reservations per facility over the last 30 days
SELECT v.name AS venue, f.name AS facility, COUNT(r.reservation_id) AS
bookings
FROM Facilities f
JOIN Venues v ON v.venue_id = f.venue_id
LEFT JOIN Reservations r
  ON r.facility_id = f.facility_id
 AND r.status <> 'cancelled'
 AND r.res_date >= CURRENT_DATE - 30
GROUP BY v.name, f.name
ORDER BY bookings DESC;

-- Subquery: pickup games that still have room
SELECT g.game_id, g.sport, f.name, g.game_date, g.slot_start
FROM PickupGames g
JOIN Facilities f ON f.facility_id = g.facility_id
WHERE g.game_date >= CURRENT_DATE
  AND (SELECT COUNT(*) FROM PickupPlayers p WHERE p.game_id = g.game_id) <
g.max_players;
```

Constraints

Rule	How the database enforces it
One reservation per student per day	`CREATE UNIQUE INDEX one_per_day ON Reservations (user_id, res_date) WHERE status <> 'cancelled';`
No double-booking past a facility's capacity	Trigger `check_capacity` (below) counts active bookings for the slot against `Facilities.capacity`
Walk-up slots (11 am, 1 pm) can't be reserved	`CHECK (slot_start IN (7, 9, 15, 17, 19))`
Valid statuses, roles and types	`CHECK (... IN (...))` on every status, role and type column
Hours make sense	`CHECK (closes_at > opens_at)` on OperatingHours

Rule	How the database enforces it
A student joins a pickup game once	`PRIMARY KEY (game_id, user_id)` on PickupPlayers
One account per email	`UNIQUE (email)` on Users

Views

```
-- Open slots today, with spots left (backs the availability grid)
CREATE VIEW todays_availability AS
SELECT f.facility_id, f.name, v.name AS venue, s.slot_start,
       f.capacity - COUNT(r.reservation_id) AS spots_left
FROM Facilities f
JOIN Venues v ON v.venue_id = f.venue_id
CROSS JOIN (VALUES (7), (9), (15), (17), (19)) AS s(slot_start)
LEFT JOIN Reservations r
  ON r.facility_id = f.facility_id AND r.res_date = CURRENT_DATE
 AND r.slot_start = s.slot_start AND r.status <> 'cancelled'
WHERE f.is_active
GROUP BY f.facility_id, f.name, v.name, f.capacity, s.slot_start
HAVING f.capacity - COUNT(r.reservation_id) > 0;
```

A second view, `approved_events_feed`, joins Events, Organizations and Facilities WHERE status = 'approved' for the public events feed.

Stored procedures and triggers

- **`check_capacity` (BEFORE INSERT trigger on Reservations).** Rejects the insert if the slot is already at the facility's capacity, if an approved event holds it, or if the user is banned, locked out or hasn't passed the quiz.
- **`apply_no_show` (AFTER UPDATE trigger on Reservations).** When status becomes 'no-show', increments `Users.no_show_count`. One no-show sets status to 'warned' and `locked_until` to a temporary lockout; three set status to 'banned'.
- **`mark_no_shows()` (stored procedure, run every 2 hours).** Sets every 'booked' reservation whose slot has ended to 'no-show', which fires `apply_no_show`.

```
CREATE FUNCTION apply_no_show() RETURNS trigger AS $$
BEGIN
  IF NEW.status = 'no-show' AND OLD.status <> 'no-show' THEN
    UPDATE Users
    SET no_show_count = no_show_count + 1,
        status = CASE WHEN no_show_count + 1 >= 3 THEN 'banned' ELSE 'warned'
END,
        locked_until = CASE WHEN no_show_count + 1 < 3 THEN now() + INTERVAL
'7 days' END
    WHERE user_id = NEW.user_id;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER no_show_penalty
AFTER UPDATE OF status ON Reservations
FOR EACH ROW EXECUTE FUNCTION apply_no_show();
```

The 7-day lockout is a placeholder; the site still lists the lockout length as a TODO.

Transactions

Joining a pickup game checks the player count and adds the player as one unit. The row lock on the game stops two students taking the last spot at the same moment; if the game is full, nothing is written. Booking a reservation uses the same pattern, locking the facility row while the capacity check and insert run.

```
BEGIN;
  SELECT max_players FROM PickupGames WHERE game_id = 7 FOR UPDATE;
  SELECT COUNT(*) FROM PickupPlayers WHERE game_id = 7;  -- app checks count
< max_players
  INSERT INTO PickupPlayers (game_id, user_id) VALUES (7, '800000001');
COMMIT;   -- game full or already joined -> ROLLBACK
```

4. Screenshots

Four of the six screens exist on the live site today; log in with `dev@sdsu.edu` / `courtconnect-dev` to reach them. Insert one image above each caption.

#	Screen	Caption	Status
1	Home	The CourtConnect home page and sign-up wall: students register with their RedID and @sdsu.edu email.	Ready to capture
2	Facility availability	The availability grid for the Aztec Recreation Center: each space's 2-hour slots show open, booked, full, walk-up or event.	Ready to capture
3	Reservation flow	The confirm dialog before booking, with the space, time, venue and the one-per-day reminder.	Ready to capture
4	Your reservations	A student's bookings with Check in and Cancel buttons and booked, checked-in or no-show badges.	Ready to capture
5	Pickup game post	Posting an open game and the list of games with spots left.	Not built yet
6	Club events / admin view	An organization's event request and the admin queue that approves it.	Not built yet

5. User Guide

Steps marked *(planned)* describe features whose tables exist in the design but whose screens are not built yet.

Student

1. **Sign up.** Enter your name, 9-digit RedID, @sdsu.edu email and a password of 8+ characters.
2. **Pass the facility quiz** *(planned)*. Complete the access form and the safety and etiquette quiz. You can't reserve until you pass.
3. **Find a court.** Pick a venue from the dropdown, such as Aztec Recreation Center or Aztec Tennis Center, then a day in the next 7.
4. **Check availability.** Each space shows its slots from 7 am to 9 pm. *Open* or *N spots* can be booked; *Walk-up* (11 am and 1 pm) is first come, first served; *Event* is held by a club.

5. **Reserve it.** Click an open slot and confirm. You get one reservation per day.
6. **Check in.** In *Your reservations*, press **Check in** from 15 minutes before your slot until it ends. Miss it and it counts as a no-show: one brings a temporary lockout, three a permanent ban.
7. **Cancel** any time before the slot starts to free it for someone else.
8. **Join a pickup game** *(planned)*. Open *Pickup games*, pick one with spots left and press **Join**, or post your own with the sport, court, time and player limit.

Club or organization

1. **Register the organization** *(planned)*. The contact user creates it with its name and type (club, fraternity, sorority) and adds members.
2. **Request a space** *(planned)*. Choose a facility, such as ENS 700 Field or ARC North Court, a date and a start and end slot, and add a title and description.
3. **Wait for approval.** The request is *pending* until an admin approves it. Approved events block those slots for students and appear on the events feed.

Admin

1. **Manage facilities** *(planned)*. Add or edit venues, opening hours and spaces, set each space's capacity, or mark a space inactive to take it off the schedule.
2. **Approve requests** *(planned)*. Review the queue of pending club events and approve or reject each one; the decision records the admin's ID.
3. **Handle no-shows** *(planned)*. View users who are warned or banned and their no-show count, and lift a lockout when appropriate.

6. AI Usage Disclosure

Where AI was used

- **This document.** Claude (Anthropic) drafted this technical document from the team's repository: the schema on the project site, the booking rules, and `js/booking.js`. It proposed the five added tables, the SQL examples, the view, triggers and transaction, and the normalization examples.
- **Code and website.** *[Team to fill in: e.g. brainstorming the feature list, starter code for the booking page, debugging, CSS.]*

What our team changed or extended

[Team to fill in with specific examples. Suggestions to check before submitting:]

- [ ] Confirm PostgreSQL and the backend choice, or replace them with what we actually use.
- [ ] Confirm the added tables (Venues, OperatingHours, OrgMembers, PickupGames, PickupPlayers) match our plan.
- [ ] Run every SQL example against our real DDL and fix anything that fails.
- [ ] Replace the placeholder 7-day lockout with our real number.
- [ ] Describe what each of us wrote or rewrote by hand.